

## CSCE 2014 - Laboratory Assignment 12

This laboratory assignment focuses on hash tables. As we will see, well designed hash tables can provide very fast access to data. In fact, this data structure is known to take "constant time"  $O(1)$  because the CPU time needed to store/retrieve data does not depend on the amount of data being stored. We can search 1,000,000 items as quickly as we can search 1,000 items. The remainder of this lab goes into the the implementation and use of hash tables in more detail.

### 1. Introduction to Hash Tables

The basic idea behind hash tables is to use the data value itself to decide **where** to store and retrieve data records. We do this by using a **hash function** that maps between data values and table locations. For example, if hash function for the string "hello" is equal to 42, we would store the string "hello" in position 42 in the table. This makes it very easy to add "hello" to the hash table, and also very easy to look up this data record at a later time.

Hash functions can be created for a wide variety of data types (strings, integers, floats, etc.) by any combination of arithmetic operations that yields a value between  $[0..SIZE-1]$  where SIZE is the size of the hash table. Hash functions are typically "many to one", meaning that there are many data values that could produce the same hash value. For example, "elephant" could also have a hash value of 42. If this happens, there is a **collision** in the hash table when we try to insert both strings in the table. We must design hash table implementations to deal with this situation.

### 2. The HashTable Class

Consider the following declaration of the HashTable class. You will see that the class has four public operations: Insert, Search, delete, and Print. You will also see that there are two private methods Hash and Hash2. The private data for the hash table is stored in two dynamically allocated arrays of integers called "Value" and "Key". The size of the hash table is in the private integer "Size".

```
//-----  
// Purpose:    Header file for the HashTable class.  
// Author:     John Gauch  
//-----  
  
#include <iostream>  
using namespace std;  
const int NONE = 0;  
const int EMPTY = -1;  
const int DELETED = -2;  
  
//-----  
// Define the HashTable class interface  
//-----  
class HashTable  
{  
public:
```

```

    // Constructors
    HashTable(int size);
    HashTable(const HashTable & ht);
    ~HashTable();

    // Methods
    bool Insert(int key, int value);
    bool Search(int key, int &value);
    bool Delete(int key);
    void Print();

private:
    // Private methods
    int Hash(int key);
    int Hash2(int index);

    // Private data
    int Size;
    int *Value;
    int *Key;
};

```

### 3. HashTable Implementation

Consider the implementation of the HashTable class below. The constructor function allocates two arrays of integers of length "Size" and initializes these arrays to EMPTY. The destructor function releases this memory when you are finished using your HashTable objects.

The "Insert" method calls the private method "Hash" to determine where to store the (key, value) pair. If this array location is already occupied, the method calls "Hash2" to search for an empty location. This collision handling technique is called **linear probing**.

The "Search" method also calls "Hash" and "Hash2" to locate the requested (key, value) pair. The Search method returns *true* if the key is found in the hash table, and *false* otherwise.

The "Delete" method first calls "Hash" and "Hash2" to locate the requested key in the hash table. If found, the key is changed to DELETED to indicate that this (key, value) pair has been removed from the table. This method also returns *true* if the key is found in the hash table, and *false* otherwise.

```

//-----
// Purpose:      Implementation of HashTable class.
// Author:       John Gauch
//-----
#include "hash.h"

//-----
// Constructor
//-----
HashTable::HashTable(int size)
{
    Size = size;
    Value = new int[Size];
    Key = new int[Size];

    for (int index=0; index < Size; index++)

```

```

    {
        Value[index] = NONE;
        Key[index] = EMPTY;
    }
}

//-----
// Copy constructor
//-----
HashTable::HashTable(const HashTable & ht)
{
    Size = ht.Size;
    Value = new int[Size];
    Key = new int[Size];

    for (int index=0; index < Size; index++)
    {
        Value[index] = ht.Value[index];
        Key[index] = ht.Key[index];
    }
}

//-----
// Destructor
//-----
HashTable::~HashTable()
{
    delete []Value;
    delete []Key;
}

//-----
// Insert method
//-----
bool HashTable::Insert(int key, int value)
{
    // Find desired key
    int index = Hash(key);
    while ((Key[index] != key) && (Key[index] != EMPTY))
        index = Hash2(index);

    // Insert value into hash table
    Value[index] = value;
    Key[index] = key;
    return true;
}

//-----
// Search method
//-----
bool HashTable::Search(int key, int &value)
{
    // Find desired key
    int index = Hash(key);
    while ((Key[index] != key) && (Key[index] != EMPTY))
        index = Hash2(index);

```

```

        // Return value from hash table
        if (Key[index] == key)
            value = Value[index];
        return (Key[index] == key);
    }

//-----
// Delete method
//-----
bool HashTable::Delete(int key)
{
    // Find desired key
    int index = Hash(key);
    while ((Key[index] != key) && (Key[index] != EMPTY))
        index = Hash2(index);

    // Delete value from hash table
    if (Key[index] == key)
    {
        Value[index] = NONE;
        Key[index] = DELETED;
        return true;
    }
    return false;
}

//-----
// Primary hash function
//-----
int HashTable::Hash(int key)
{
    return key % Size;
}

//-----
// Secondary hash function
//-----
int HashTable::Hash2(int index)
{
    return (index+1) % Size;
}

//-----
// Print function for debugging
//-----
void HashTable::Print()
{
    cout << "Index\t" << "Value\t" << "Key\n";
    for (int index=0; index < Size; index++)
        cout << index << "\t"
            << Value[index] << "\t"
            << Key[index] << "\n";
}

//-----
// Main program.
//-----

```

```
int main()
{
    // Initialize random number generator
    srand(time(NULL));

    // ADD HERE
}
```

#### 4. Testing the HashTable Class

Copy and paste the class declaration above into a file called "hash.h" and the class implementation into a file called "hash.cpp" and compile and run this program in your IDE.

Now extend the main function to create a HashTable object of SIZE=20. Then write a loop to insert COUNT=10 random (key,value) pairs into the hash table. To do this, you should use "key = random() % MAX" and "value = random() % MAX" where MAX=1000. Call the "Print" method to see what your hash table contains. You should see different numbers every time you run the program because the random number generator is seeded with the current time.

Cut and paste your new main function below:

```
int main()
{
    srand(time(NULL));

    const int SIZE = 20;
    const int COUNT = 10;
    const int MAX = 1000;

    HashTable ht(SIZE);

    for (int i = 0; i < COUNT; i++)
    {
        int key = random() % MAX;
        int value = random() % MAX;
        ht.Insert(key, value);
    }

    ht.Print();
    return 0;
}
```

Cut and paste your program output below:

| Index | Value | Key |
|-------|-------|-----|
| 0     | 703   | 920 |
| 1     | 0     | -1  |
| 2     | 0     | -1  |
| 3     | 247   | 743 |
| 4     | 0     | -1  |
| 5     | 447   | 205 |
| 6     | 276   | 626 |
| 7     | 0     | -1  |
| 8     | 0     | -1  |
| 9     | 413   | 629 |
| 10    | 0     | -1  |
| 11    | 616   | 651 |
| 12    | 0     | -1  |
| 13    | 0     | -1  |
| 14    | 0     | -1  |
| 15    | 399   | 495 |
| 16    | 331   | 555 |
| 17    | 753   | 615 |
| 18    | 0     | -1  |
| 19    | 60    | 719 |

## 5. Detecting Collisions

Edit your program to prompt the user for three input values: SIZE, COUNT, and MAX and use these values in your main function. Repeat the experiment with SIZE=20, COUNT=15, and MAX=1000. When you print the hash table, you may see several consecutive hash table entries where the last digits of the keys are the same. These are the result of collisions (where two keys hashed to the the same array location).

Go into your code and edit the "Hash2" function to print out a message every time it is called to handle a collision. Run an experiment with SIZE=40, COUNT=15, and MAX=1000. There should be fewer collisions than you had above. Run another experiment with SIZE=40, COUNT=30, and MAX=1000. You should see an increase in the number of collisions.

Run a series of experiments with SIZE=40 and MAX=1000 to fill in the table below. Be careful to keep COUNT < SIZE, in your experiments or your program may go into an infinite loop. If you do end up with an infinite loop, kill your program quickly to save CPU cycles.

| COUNT | COLLISIONS |
|-------|------------|
| 5     | -- 0       |
| 10    | -- 0       |
| 15    | -- 8       |
| 20    | -- 10      |
| 25    | -- 14      |
| 30    | -- 12      |
| 35    | -- 80      |

## 6. Fixing the Insert Method

As noted above, it is possible for the "Insert" method to go into an infinite loop if the table runs out of empty locations. There are several ways to fix this problem.

- You can add an insertion counter to the class and only insert new data to the hash table when this count is less than the SIZE.
- You can modify the while loops to stop when they wrap around to the starting position of the search.

- You can modify the while loops to stop after looking at SIZE locations in the hash table.

Choose one of the methods above, and go into your code to fix the infinite loop problem. When you think you have it working, test your code with SIZE=40 and COUNT=50 to verify that it works properly. If you do end up with an infinite loop, kill your program quickly to save CPU cycles.

Cut and paste your new "Insert" method below:

```
bool HashTable::Insert(int key, int value) {
    // Find desired key
    int index = Hash(key);
    int checked = 0;

    while ((Key[index] != key) && (Key[index] != EMPTY && (checked < Size))) {
        index = Hash2(index);
        checked++;
    }

    // Table is full, insertion failed
    if (checked == Size) {
        cout << "Insert failed: table is full\n";
        return false;
    }
    // Insert value into hash table
    Value[index] = value;
    Key[index] = key;
    return true;
}
```

Cut and paste your new program output below:

Insert failed: table is full  
Insert failed: table is full  
Insert failed: table is full  
Insert failed: table is full  
Insert failed: table is full  
Insert failed: table is full  
Insert failed: table is full

| Index | Value | Key |
|-------|-------|-----|
| 0     | 342   | 400 |
| 1     | 552   | 481 |
| 2     | 513   | 442 |
| 3     | 559   | 202 |
| 4     | 81    | 604 |
| 5     | 307   | 601 |
| 6     | 509   | 842 |
| 7     | 382   | 644 |
| 8     | 248   | 402 |
| 9     | 507   | 369 |
| 10    | 944   | 489 |
| 11    | 205   | 51  |
| 12    | 786   | 132 |
| 13    | 958   | 398 |
| 14    | 251   | 374 |
| 15    | 263   | 105 |
| 16    | 787   | 216 |
| 17    | 289   | 142 |
| 18    | 48    | 138 |
| 19    | 182   | 499 |
| 20    | 943   | 620 |
| 21    | 67    | 581 |
| 22    | 340   | 901 |
| 23    | 253   | 663 |
| 24    | 742   | 183 |
| 25    | 260   | 303 |
| 26    | 590   | 426 |
| 27    | 880   | 66  |
| 28    | 107   | 707 |
| 29    | 351   | 549 |
| 30    | 732   | 389 |
| 31    | 832   | 865 |
| 32    | 269   | 831 |
| 33    | 150   | 633 |
| 34    | 17    | 474 |
| 35    | 585   | 113 |
| 36    | 152   | 876 |
| 37    | 594   | 990 |
| 38    | 628   | 478 |
| 39    | 355   | 592 |

Total collisions: 413

## 7. Submit Work

When you have completed all of the steps to this lab:

- At the bottom of the page enter your name and UAID and click "Submit Lab".
- Print the resulting lab report web page to a PDF file.
- Login to Blackboard and go to the labs section.
- Upload your PDF file for grading and feedback.

Enter your FIRST name:

Skylar

Enter your LAST name:

Miles

Enter your UAID number:

300349320

Submit Lab